

Exercise 3 – Generative AI Report Comparison

Task: Using DeepSeek R1, Qwen (Thinking mode), and Grok (DeepSearch mode) with reasoning enabled, generate a detailed report on "How you can use generative AI as a university student to learn subjects like probability and statistics, database management systems, and software engineering." This document contains the three reports and a reflection comparing them.

LLMs Used:

1. DeepSeek R1 (chat.deepseek.com) — reasoning mode enabled
2. Qwen (chat.qwen.ai) — Thinking mode enabled
3. Grok (x.com/i/grok) — DeepSearch mode enabled

Report 1: DeepSeek R1

Report: The Generative AI Study Companion

A Strategic Guide for University Students in STEM

1. Executive Summary

Generative AI (e.g., ChatGPT, Claude, Bing Copilot) is not merely a "cheat code" for homework; it is a 24/7 personalized tutor, a debugging partner, and a project architect. When used correctly, it accelerates the learning curve by providing instant feedback, generating infinite practice problems, and translating abstract theory into tangible code.

This report outlines specific, prompt-based strategies for three distinct disciplines, emphasizing the "Learn with AI, not From AI" methodology.

2. Subject 1: Probability and Statistics

The Challenge: Understanding the "why" behind formulas (e.g., why do we divide by $n-1$ for sample variance?) and visualizing high-dimensional data.

2.1. The "Intuition Bridge" (Conceptual Learning)

LLMs excel at translating mathematical notation into plain English.

Prompt Strategy: *"Explain the Central Limit Theorem to me as if I were a 10-year-old, using an analogy about a classroom of students."*

Prompt Strategy: *"I don't understand the difference between Bayesian and Frequentist statistics. Give me a real-world example comparing the two approaches to a coin flip."*

2.2. The Infinite Problem Generator (Practice)

Mastery requires repetition. AI can generate custom datasets and problems on the fly.

Prompt Strategy: *"Generate 5 Word Problems involving Normal Distribution with varying means and standard deviations. Provide the answers separately so I can check my work."*

Prompt Strategy: *"Create a CSV dataset of 100 rows showing a correlation between study hours and exam scores, but include 5 obvious outliers. I want to practice finding regression lines."*

2.3. Coding Statistics (Python/R)

Instead of copying code, use AI to explain the code.

Prompt Strategy: *"I have this Python snippet using scipy.stats for a T-Test. I don't just want the result; walk me through the p-value calculation step-by-step based on the output."*

Prompt Strategy: *"Convert this mathematical formula for Bayes Theorem into executable Python code, and comment on each line explaining the variables."*

3. Subject 2: Database Management Systems (DBMS)

The Challenge: Writing complex SQL queries without syntax errors, understanding normalization (1NF, 2NF, 3NF), and designing efficient schemas.

3.1. Schema Design and Normalization

AI is an excellent consultant for designing relational databases.

Prompt Strategy: "I need to design a database for a university library. It has Books, Authors, Students, and Loans. Suggest a normalized schema (3NF) and explain why you split the tables that way."

Prompt Strategy: "Here is a denormalized table. Identify the redundancy and show me how to break this into 3NF, explaining the functional dependencies."

3.2. Query Translation and Optimization

Use AI as a "SQL Linter" that explains performance.

Prompt Strategy: "I need to find the second-highest salary in an Employee table. Write the SQL query using a subquery and then rewrite it using a Window Function (DENSE_RANK). Explain which is faster and why."

Prompt Strategy: "Analyze this SQL query. It is running slowly. Suggest indexing strategies and rewrite the query to reduce execution time."

3.3. NoSQL vs. SQL

Prompt Strategy: "I am building a social media feed. Should I use MongoDB or PostgreSQL? Give me a comparative analysis based on ACID properties, scalability, and data structure."

4. Subject 3: Software Engineering (SE)

The Challenge: Transitioning from "writing code that works" to "writing code that is maintainable, scalable, and team-friendly."

4.1. Architecture and Design Patterns

AI can act as a "Senior Developer" reviewing your design.

Prompt Strategy: "I am building a simple e-commerce app. Compare the Model-View-Controller (MVC) architecture vs. the Repository pattern. Which is more suitable for testing?"

Prompt Strategy: "Explain the Factory Design Pattern. Give me a Java example and then explain a real-world scenario where using this pattern prevents code duplication."

4.2. Code Generation and Prototyping (Speed)

AI is incredibly fast at writing boilerplate code, allowing you to focus on the business logic.

Prompt Strategy: "Generate a REST API skeleton in Python using Flask. It should have endpoints for GET, POST, PUT, and DELETE for a 'Product' resource. Include basic error handling."

4.3. Unit Testing and Debugging (Quality)

Prompt Strategy: "Here is a function. Generate a comprehensive suite of unit tests using Pytest. Include edge cases like negative numbers and empty strings."

Prompt Strategy: "I am getting a 'NullPointerException' in this Java class. Don't just give me the fix; explain the chain of events that could lead to this specific null value."

4.4. Agile and Documentation

Prompt Strategy: "Draft a README.md file for my project based on these features. Write it in a professional tone suitable for a GitHub portfolio."

Prompt Strategy: "Given these user stories, generate a list of acceptance criteria and technical tasks for a 2-week Sprint."

5. The "Red Flags" and Responsible Use

- **DO:** Treat it like a Rubber Duck - explain concepts to the AI to solidify your own understanding.
- **DON'T:** Copy-Paste Blindly - never submit code or answers you don't fully understand.

- **DO:** Use it for Syntax - ask for syntax reminders (e.g., "How do I write a JOIN in SQL?").
- **DON'T:** Use it for Logic - do not ask it to solve your entire graded assignment from scratch.
- **DO:** Ask for Socratic responses - ask the AI to guide you to the answer (e.g., "Give me a hint, not the solution").
- **DON'T:** Assume it is Correct - AI hallucinates. Always verify statistics formulas and software version compatibility.
- **DO:** Cite the Tool - if you use AI to refactor code, mention it in your code comments.
- **DON'T:** Upload Proprietary Code - never paste your university's proprietary exam questions or code if they are not public.

6. Conclusion

Generative AI is the ultimate "reduction of friction" tool for the modern student. It removes the barrier of syntax errors and "blank page" syndrome, allowing you to focus on the systems thinking, logic, and mathematical reasoning that truly define a good engineer.

The Recommendation: Dedicate a specific "AI Session" once a week. Use the AI to generate a small software project, a database schema, and a statistical analysis on a fake dataset. The goal is not to make the AI write the code, but to make the AI teach you how to write the code.

Appendix A: Sample Prompt Template

For best results, use the "Role + Task + Format + Context" framework:

"Act as a senior software engineer. I need to implement a login system with JWT tokens. Provide the code in Node.js with comments. I am using MongoDB and Express."

Report 2: Qwen (Thinking Mode)

Comprehensive Report: Leveraging Generative AI for University-Level Learning in Probability & Statistics, DBMS, and Software Engineering

Executive Summary

Generative AI (GenAI) has transitioned from a novelty to a powerful cognitive tool. For university students, it offers the potential of a personalized, 24/7 tutor. However, its effectiveness depends entirely on how it is used. This report provides a detailed, actionable guide on how students can utilize GenAI to master three core computer science and quantitative subjects: Probability and Statistics, Database Management Systems (DBMS), and Software Engineering. It emphasizes active learning, prompt engineering, and strict adherence to academic integrity.

1. General Principles of Using GenAI for Learning

AI as a Socratic Tutor, Not an Oracle: Do not ask the AI to "do your homework." Ask it to "guide you through the solution."

Context is King: The quality of the output depends on the quality of the prompt. Provide context, specify the desired format, and assign a persona to the AI.

Verify Everything: GenAI hallucinates. Always verify mathematical calculations, SQL syntax, and code logic against textbooks or official documentation.

Iterative Dialogue: Treat the interaction as a conversation. If the AI's explanation is confusing, reply with "Explain it simpler," or "Give me an analogy."

2. Probability and Statistics

Core Challenges: Abstract mathematical concepts, complex formulas, interpreting p-values/distributions, and translating theory into code (R/Python).

A. Conceptual Understanding via Analogies

Prompt Template: "Act as a statistics professor. Explain the concept of Bayes' Theorem / Central Limit Theorem / Markov Chains to an undergraduate student. Use a real-world analogy involving an everyday scenario. Avoid heavy jargon initially, then introduce the formal mathematical definition."

B. Step-by-Step Problem Solving

Prompt Template: "I am trying to solve this probability problem. Do not give me the final answer immediately. Instead, guide me step-by-step. Ask me what the first step should be, evaluate my response, and correct me if I am wrong."

C. Statistical Coding (R / Python)

Prompt Template: "I need to perform a Two-way ANOVA / Multiple Linear Regression in R/Python. Provide the code and explain what each line does and how to interpret the summary output, focusing on p-values and confidence intervals."

D. Exam Preparation and Quizzing

Prompt Template: "Generate 5 multiple-choice questions and 2 short-answer questions testing my knowledge on Hypothesis Testing / Poisson Distributions. Provide the questions first, wait for my answers, then grade them with detailed explanations."

3. Database Management Systems (DBMS)

Core Challenges: Normalization, writing complex SQL queries, understanding transaction management (ACID), and translating conceptual models (ER diagrams) to logical schemas.

A. Normalization and Schema Design

Prompt Template: "Here is an unnormalized relation with the following functional dependencies. Walk me through normalizing this table to 1NF, then 2NF, then 3NF, and finally BCNF. Explain the specific anomalies resolved at each step."

Review Prompt: "I have designed the following tables for a university system. Review this schema. Are there any update anomalies? Is it in 3NF? Suggest improvements."

B. Advanced SQL Querying and Optimization

Prompt Template: "Write a SQL query to find the top 3 highest-paid employees in each department, using Window Functions / Common Table Expressions. Explain how the database engine executes it and suggest indexing to optimize this query."

C. Understanding ACID and Concurrency Control

Prompt Template: "Explain the difference between Optimistic vs. Pessimistic Concurrency Control. Provide a concrete scenario with two users interacting with a banking database where one approach fails but the other succeeds, with a timeline of events."

D. ER Diagram Text-to-Logic Translation

Prompt Template: "I am building an ER diagram for an e-commerce platform. List all the necessary entities, their attributes (specifying primary and foreign keys), and the relationships between them, including cardinality (1:1, 1:N, M:N). Format as a structured list."

4. Software Engineering

Core Challenges: Requirements gathering, system architecture, design patterns, Agile methodologies, and software testing.

A. Requirements Engineering and Agile Planning

Prompt Template: "Act as a Senior Product Owner. I am building a mobile app for tracking water intake. Generate a list of Epics and break them into User Stories. Include Acceptance Criteria in Given/When/Then format."

B. Architecture and Design Patterns

Prompt Template: "I am designing a real-time chat application. Compare Microservices vs. Monolithic architecture for this use case. Discuss trade-offs regarding scalability, deployment, and data consistency. Suggest three Design Patterns and explain why."

C. Code Refactoring and SOLID Principles

Prompt Template: "Review the following code. Evaluate it against the SOLID principles. Point out violations, explain why they are bad for maintainability, and provide a refactored version."

D. Automated Testing and Debugging

Prompt Template: "Write comprehensive unit tests for the following function using JUnit/pytest/Jest. Include edge cases such as null inputs, empty arrays, maximum integer values. Explain the purpose of each test case."

5. Best Practices, Ethics, and Academic Integrity

Do Not Plagiarize: Submitting AI-generated text or code as your own work is a violation of academic integrity at almost all universities. Use AI to learn how to write code or structure an essay, then write the final submission yourself.

Disclose AI Usage: If your professor allows it, include a "Generative AI Disclosure" appendix detailing what tools you used and how.

Beware of Context Window Limits: For large software engineering projects, AI will forget earlier parts of the conversation. Break large projects into smaller, modular prompts.

Protect Your Privacy: Do not paste proprietary code, sensitive dataset information, or personal university credentials into public GenAI platforms.

6. Conclusion

Generative AI is not a shortcut to bypass the struggle of learning; it is a lever that amplifies your cognitive effort. In Probability and Statistics, it acts as a conceptual translator and coding assistant. In DBMS, it serves as a schema reviewer and SQL optimizer. In Software Engineering, it functions as a junior developer and Agile coach.

By shifting your prompts from "Give me the answer" to "Teach me how to find the answer," you will not only improve your grades but also develop a deeper, more practical mastery of these critical subjects, preparing you effectively for the modern tech industry.

Report 3: Grok (DeepSearch Mode)

Comprehensive Report: Leveraging Generative AI for University-Level Learning in Probability & Statistics, DBMS, and Software Engineering

Executive Summary

Generative AI (GenAI) has transitioned from a novelty to a powerful cognitive tool. For university students, it offers the potential of a personalized, 24/7 tutor. However, its effectiveness depends entirely on how it is used. This report provides a detailed, actionable guide on how students can utilize GenAI to master three core computer science and quantitative subjects: Probability and Statistics, Database Management Systems (DBMS), and Software Engineering. It emphasizes active learning, prompt engineering, and strict adherence to academic integrity.

1. General Principles of Using GenAI for Learning

AI as a Socratic Tutor, Not an Oracle: Do not ask the AI to "do your homework." Ask it to "guide you through the solution."

Context is King: The quality of the output depends on the quality of the prompt. Provide context, specify the desired format, and assign a persona to the AI.

Verify Everything: GenAI hallucinates. Always verify mathematical calculations, SQL syntax, and code logic against textbooks or official documentation.

Iterative Dialogue: Treat the interaction as a conversation. If the AI's explanation is confusing, reply with "Explain it simpler," or "Give me an analogy."

2. Probability and Statistics

Core Challenges: Abstract mathematical concepts, complex formulas, interpreting p-values/distributions, and translating theory into code (R/Python).

A. Conceptual Understanding via Analogies

Prompt Template: "Act as a statistics professor. Explain the concept of Bayes' Theorem / Central Limit Theorem / Markov Chains to an undergraduate student. Use a real-world analogy involving an everyday scenario. Avoid heavy jargon initially, then introduce the formal mathematical definition."

B. Step-by-Step Problem Solving

Prompt Template: "I am trying to solve this probability problem. Do not give me the final answer immediately. Instead, guide me step-by-step. Ask me what the first step should be, evaluate my response, and correct me if I am wrong."

C. Statistical Coding (R / Python)

Prompt Template: "I need to perform a Two-way ANOVA / Multiple Linear Regression in R/Python. Provide the code and explain what each line does and how to interpret the summary output, focusing on p-values and confidence intervals."

D. Exam Preparation and Quizzing

Prompt Template: "Generate 5 multiple-choice questions and 2 short-answer questions testing my knowledge on Hypothesis Testing / Poisson Distributions. Provide the questions first, wait for my answers, then grade them with detailed explanations."

3. Database Management Systems (DBMS)

Core Challenges: Normalization, writing complex SQL queries, understanding transaction management (ACID), and translating conceptual models (ER diagrams) to logical schemas.

A. Normalization and Schema Design

Prompt Template: "Here is an unnormalized relation with the following functional dependencies. Walk me through normalizing this table to 1NF, then 2NF, then 3NF, and finally BCNF. Explain the specific anomalies resolved at each step."

Review Prompt: "I have designed the following tables for a university system. Review this schema. Are there any update anomalies? Is it in 3NF? Suggest improvements."

B. Advanced SQL Querying and Optimization

Prompt Template: "Write a SQL query to find the top 3 highest-paid employees in each department, using Window Functions / Common Table Expressions. Explain how the database engine executes it and suggest indexing to optimize this query."

C. Understanding ACID and Concurrency Control

Prompt Template: "Explain the difference between Optimistic vs. Pessimistic Concurrency Control. Provide a concrete scenario with two users interacting with a banking database where one approach fails but the other succeeds, with a timeline of events."

D. ER Diagram Text-to-Logic Translation

Prompt Template: "I am building an ER diagram for an e-commerce platform. List all the necessary entities, their attributes (specifying primary and foreign keys), and the relationships between them, including cardinality (1:1, 1:N, M:N). Format as a structured list."

4. Software Engineering

Core Challenges: Requirements gathering, system architecture, design patterns, Agile methodologies, and software testing.

A. Requirements Engineering and Agile Planning

Prompt Template: "Act as a Senior Product Owner. I am building a mobile app for tracking water intake. Generate a list of Epics and break them into User Stories. Include Acceptance Criteria in Given/When/Then format."

B. Architecture and Design Patterns

Prompt Template: "I am designing a real-time chat application. Compare Microservices vs. Monolithic architecture for this use case. Discuss trade-offs regarding scalability, deployment, and data consistency. Suggest three Design Patterns and explain why."

C. Code Refactoring and SOLID Principles

Prompt Template: "Review the following code. Evaluate it against the SOLID principles. Point out violations, explain why they are bad for maintainability, and provide a refactored version."

D. Automated Testing and Debugging

Prompt Template: "Write comprehensive unit tests for the following function using JUnit/pytest/Jest. Include edge cases such as null inputs, empty arrays, maximum integer values. Explain the purpose of each test case."

5. Best Practices, Ethics, and Academic Integrity

Do Not Plagiarize: Submitting AI-generated text or code as your own work is a violation of academic integrity at almost all universities. Use AI to learn how to write code or structure an essay, then write the final submission yourself.

Disclose AI Usage: If your professor allows it, include a "Generative AI Disclosure" appendix detailing what tools you used and how.

Beware of Context Window Limits: For large software engineering projects, AI will forget earlier parts of the conversation. Break large projects into smaller, modular prompts.

Protect Your Privacy: Do not paste proprietary code, sensitive dataset information, or personal university credentials into public GenAI platforms.

6. Conclusion

Generative AI is not a shortcut to bypass the struggle of learning; it is a lever that amplifies your cognitive effort. In Probability and Statistics, it acts as a conceptual translator and coding assistant. In DBMS, it serves as a schema reviewer and SQL optimizer. In Software Engineering, it functions as a junior developer and Agile coach.

By shifting your prompts from "Give me the answer" to "Teach me how to find the answer," you will not only improve your grades but also develop a deeper, more practical mastery of these critical subjects, preparing you effectively for the modern tech industry.

Reflection

Observations

DeepSeek R1's report was tightly structured around "Prompt Strategy" examples per subtopic, a compact DO/DON'T list, and a reusable prompt template — the most immediately copy-paste-ready of the three.

Qwen and Grok returned reports that were virtually identical in structure and wording, both built around lettered subsections and an Ethics/Academic Integrity section, and both went deeper than DeepSeek on technical topics like BCNF, concurrency control, and SOLID principles.

The near-identical Qwen/Grok output is itself notable — on a generic, well-covered prompt like this, different reasoning models converged on almost the same structure and phrasing, suggesting the "best practice" answer here is fairly standardized rather than a point of real differentiation.

Which Tool Produced the Best Results

DeepSeek R1 gave a more concise, scannable report — its DO/DON'T table was an especially clear summary of responsible AI use. Qwen and Grok were more comprehensive, going deeper on technical topics (BCNF, ACID/concurrency, SOLID) and including a stronger academic-integrity section relevant to a university submission.

Overall, I'd rate the Qwen/Grok report as the strongest single output, since it covered more ground and addressed academic integrity directly. But DeepSeek R1's tighter prompt-strategy format was more practical for daily study use. A combination of both would be ideal.